

PYTHON PROJECT DOCS

project.py

Sentiment Analysis CLI

A command-line tool that classifies text as POSITIVE or NEGATIVE using a DistilBERT model fine-tuned on SST-2. Supports both inline text arguments and batch file input with zero boilerplate.

MODEL

DistilBERT SST-2

LANGUAGE

Python 3.10+

DATE

May 24, 2026

VERSION

1.0.0

Table of Contents

- | | |
|-------------------|----------------------------|
| 1 | Summary & Dependencies |
| 2 | Full Source Code |
| 3 | Function Reference |
| 4 | Usage Guide |
| 5 | Appendix — Markdown Source |
-

1 Summary & Dependencies

project.py is a self-contained command-line interface for running sentiment analysis on arbitrary English text. It wraps the Hugging Face **Transformers** *pipeline* API around the `distilbert-base-uncased-finetuned-sst-2-english` checkpoint — a DistilBERT model fine-tuned on the Stanford Sentiment Treebank v2 dataset — to predict binary POSITIVE/NEGATIVE labels with a confidence score. The pipeline is initialised lazily on first use via a module-level singleton, so repeated calls within the same process pay no reload overhead. Input can be supplied as one or more inline `--text` arguments or as a UTF-8 plain-text file (one sentence per line) via `--input-file`; both sources can be combined in a single invocation.

Dependencies

Package	Role	Notes
transformers	Core ML pipeline & tokeniser	<code>pip install transformers</code>
torch / tensorflow	Backend tensor engine	One of the two required
argparse	CLI argument parsing	Python stdlib
pathlib	File-path abstraction	Python stdlib
os	Environment variable access	Python stdlib

2 Full Source Code

project.py — 98 lines

```

1  from __future__ import annotations
2
3  import argparse
4  import os
5  from pathlib import Path
6  from typing import Iterable
7
8  from transformers import Pipeline, pipeline
9
10 MODEL_NAME = "distilbert-base-uncased-finetuned-sst-2-english"
11 HF_TOKEN_ENV = "HF_TOKEN"
12
13 _classifier: Pipeline | None = None
14
15
16 def get_classifier() -> Pipeline:
17     global _classifier
18     if _classifier is None:
19         auth_token = os.environ.get(HF_TOKEN_ENV)
20         pipeline_kwargs = {
21             "model": MODEL_NAME,
22             "tokenizer": MODEL_NAME,
23             "return_all_scores": False,
24         }
25         if auth_token:
26             pipeline_kwargs["use_auth_token"] = auth_token
27
28         _classifier = pipeline(
29             "sentiment-analysis",
30             **pipeline_kwargs,
31         )
32     return _classifier
33
34
35 def predict_sentiment(texts: Iterable[str]) -> list[dict[str, object]]:
36     classifier = get_classifier()
37     return classifier(list(texts), truncation=True)
38
39
40 def format_prediction(text: str, prediction: dict[str, object]) -> str:
41     label = prediction.get("label", "UNKNOWN")
42     score = prediction.get("score", 0.0)
43     return f"{label} ({score:.4f}) - {text}"

```

```
44
45
46 def load_texts_from_file(path: Path) -> list[str]:
47     text = path.read_text(encoding="utf-8").strip()
48     if not text:
49         return []
50     return [line.strip() for line in text.splitlines() if line.strip()]
51
52
53 def build_parser() -> argparse.ArgumentParser:
54     parser = argparse.ArgumentParser(
55         description="Sentiment analysis using a pretrained BERT-based
56         model.",
57         formatter_class=argparse.ArgumentDefaultsHelpFormatter,
58     )
59     parser.add_argument(
60         "--text",
61         "-t",
62         nargs="+",
63         help="One or more text strings to analyze.",
64     )
65     parser.add_argument(
66         "--input-file",
67         "-i",
68         type=Path,
69         help="Path to a UTF-8 text file with one sentence per line.",
70     )
71     return parser
72
73 def main() -> None:
74     parser = build_parser()
75     args = parser.parse_args()
76
77     texts: list[str] = []
78     if args.text:
79         texts.extend(args.text)
80
81     if args.input_file:
82         if not args.input_file.exists():
83             raise SystemExit(f"Error: input file does not exist:
84             {args.input_file}")
85         texts_from_file = load_texts_from_file(args.input_file)
86         texts.extend(texts_from_file)
87
88     if not texts:
89         parser.print_help()
```

89	print("\nExample:\n python project.py -t \"I love this movie\" \"This is terrible\")
90	raise SystemExit(0)
91	
92	predictions = predict_sentiment(texts)
93	for text, prediction in zip(texts, predictions):
94	print(format_prediction(text, prediction))
95	
96	
97	if __name__ == "__main__":
98	main()

3 Function Reference

`get_classifier()`

→ Pipeline

Implements a lazy singleton pattern for the HuggingFace pipeline. On the first call it reads the `HF_TOKEN` environment variable; if present, the token is forwarded as `use_auth_token` to support private or gated models. The pipeline is configured for *sentiment-analysis* with `return_all_scores=False` so only the winning label is returned. Subsequent calls skip initialisation and return the cached object, making repeated inference cheap.

`predict_sentiment(texts)`

→ list[dict[str, object]]

Accepts any iterable of strings (generator, list, tuple, etc.) and materialises it into a list before forwarding it to the pipeline. Token truncation is always enabled via `truncation=True`, which prevents runtime errors on inputs longer than the model's 512-token context window. Returns a list of dicts, one per input, each containing `label` (str) and `score` (float).

`format_prediction(text, prediction)`

→ str

A pure formatting function with no side effects. Extracts `label` and `score` from the prediction dict with safe defaults (`UNKNOWN / 0.0`) so the function never raises on malformed pipeline output. The score is rendered to four decimal places, giving callers a stable, machine-parseable output format convenient for downstream piping or logging.

`load_texts_from_file(path)`

→ list[str]

Reads the entire file at *path* as UTF-8, splits on newlines, strips leading/trailing whitespace from every line, and discards blank lines. Returns an empty list for files that are empty or contain only whitespace, allowing callers to detect the no-input condition without raising an exception. The function does not open streams itself; it delegates to `Path.read_text`, which guarantees the file handle is always closed.

`build_parser()`

→ argparse.ArgumentParser

Constructs and returns the argument parser used by `main()`. Separating parser construction into its own function makes the CLI interface independently testable without executing any I/O. The `--text` flag uses `nargs='+'` to accept a variable number of space-separated strings in one invocation, while `--input-file` accepts a `Path` object that is validated for existence at runtime by `main()`.

`main()`

→ None

The script's entry point, invoked by the `if __name__ == '__main__':` guard. It orchestrates argument parsing, input aggregation from both CLI text and file sources, and output formatting. If neither source provides any text, it prints the help message plus a quick example and exits with code 0. File existence is validated before reading so that missing paths yield a clear error message rather than a bare Python traceback.

4 Usage Guide

Installation

```
pip install transformers torch
# or with TensorFlow backend:
pip install transformers tensorflow
```

Analyse inline text (single string)

```
$ python project.py -t "I love this movie!"
```

```
POSITIVE (0.9998) - I love this movie!
```

Analyse multiple strings at once

```
$ python project.py -t "Great film!" "Absolutely terrible."
```

```
POSITIVE (0.9997) - Great film!
NEGATIVE (0.9994) - Absolutely terrible.
```

Batch-process from a file

```
$ python project.py -i reviews.txt
```

```
POSITIVE (0.9991) - Loved every minute.
NEGATIVE (0.9988) - Total waste of time.
```

Combine inline text and file

```
$ python project.py -t "Quick test" -i reviews.txt
```

```
POSITIVE (0.9876) - Quick test
...file lines follow...
```

Use a private HuggingFace model

```
$ HF_TOKEN=hf_xxxx python project.py -t "Hello"
```

```
POSITIVE (0.8732) - Hello
```

Show help

```
$ python project.py --help
```

```
usage: project.py [-h] [--text TEXT [TEXT ...]] [--input-file INPUT_FILE]
...
```

■ **HF_TOKEN** is optional. Set it only when accessing private or gated models on the Hugging Face Hub. For the default public `distilbert-base-uncased-finetuned-sst-2-english` model no token is required.

5 Appendix — Markdown Source

The following is the complete Markdown document used as the basis for this PDF.

1	# Sentiment Analysis CLI – project.py
2	
3	Date: May 24, 2026
4	
5	Description: A command-line sentiment analysis tool that classifies input
6	text as POSITIVE or NEGATIVE using a pre-trained DistilBERT model hosted on
7	Hugging Face. Supports inline text arguments and batch processing from a file.
8	
9	---
10	
11	## Table of Contents
12	
13	1. Summary & Dependencies
14	2. Full Source Code
15	3. Function / Class Reference
16	4. Usage
17	
18	---
19	
20	## 1. Summary & Dependencies
21	
22	`project.py` is a lightweight CLI wrapper around Hugging Face Transformers'
23	`pipeline` API. It loads the `distilbert-base-uncased-finetuned-sst-2-english`
24	model on first use (lazy singleton), classifies one or more text strings, and
25	prints label + confidence score to stdout.
26	
27	Dependencies: `transformers`, `torch` (or `tensorflow`), `argparse`
28	(stdlib), `pathlib` (stdlib).
29	
30	---
31	
32	## 2. Full Source Code
33	
34	```python
35	from __future__ import annotations
36	
37	import argparse
38	import os
39	from pathlib import Path
40	from typing import Iterable
41	
42	from transformers import Pipeline, pipeline
43	
44	MODEL_NAME = "distilbert-base-uncased-finetuned-sst-2-english"

```

45 HF_TOKEN_ENV = "HF_TOKEN"
46
47 _classifier: Pipeline | None = None
48
49
50 def get_classifier() -> Pipeline:
51     global _classifier
52     if _classifier is None:
53         auth_token = os.environ.get(HF_TOKEN_ENV)
54         pipeline_kwargs = {
55             "model": MODEL_NAME,
56             "tokenizer": MODEL_NAME,
57             "return_all_scores": False,
58         }
59         if auth_token:
60             pipeline_kwargs["use_auth_token"] = auth_token
61
62         _classifier = pipeline(
63             "sentiment-analysis",
64             **pipeline_kwargs,
65         )
66         return _classifier
67
68
69 def predict_sentiment(texts: Iterable[str]) -> list[dict[str, object]]:
70     classifier = get_classifier()
71     return classifier(list(texts), truncation=True)
72
73
74 def format_prediction(text: str, prediction: dict[str, object]) -> str:
75     label = prediction.get("label", "UNKNOWN")
76     score = prediction.get("score", 0.0)
77     return f"{label} ({score:.4f}) - {text}"
78
79
80 def load_texts_from_file(path: Path) -> list[str]:
81     text = path.read_text(encoding="utf-8").strip()
82     if not text:
83         return []
84     return [line.strip() for line in text.splitlines() if line.strip()]
85
86
87 def build_parser() -> argparse.ArgumentParser:
88     parser = argparse.ArgumentParser(
89         description="Sentiment analysis using a pretrained BERT-based model.",
90         formatter_class=argparse.ArgumentDefaultsHelpFormatter,
91     )
92     parser.add_argument(
93         "--text",
94         "-t",

```

```

95     nargs="+",
96     help="One or more text strings to analyze.",
97 )
98 parser.add_argument(
99     "--input-file",
100    "-i",
101    type=Path,
102    help="Path to a UTF-8 text file with one sentence per line.",
103 )
104 return parser
105
106
107 def main() -> None:
108     parser = build_parser()
109     args = parser.parse_args()
110
111     texts: list[str] = []
112     if args.text:
113         texts.extend(args.text)
114
115     if args.input_file:
116         if not args.input_file.exists():
117             raise SystemExit(f"Error: input file does not exist: {args.input_file}")
118         texts_from_file = load_texts_from_file(args.input_file)
119         texts.extend(texts_from_file)
120
121     if not texts:
122         parser.print_help()
123         print("\nExample:\n python project.py -t \"I love this movie\" \"This is terrible\"")
124         raise SystemExit(0)
125
126     predictions = predict_sentiment(texts)
127     for text, prediction in zip(texts, predictions):
128         print(format_prediction(text, prediction))
129
130
131 if __name__ == "__main__":
132     main()
133
134 ```
135
136 ---
137
138 ## 3. Function / Class Reference
139
140 ### `get_classifier() -> Pipeline`
141 Lazy singleton loader for the HuggingFace pipeline. Checks for an optional
142 `HF_TOKEN` environment variable for private-model authentication.
143
144 ### `predict_sentiment(texts) -> list[dict]`

```

```

145 Accepts any iterable of strings, converts it to a list, and runs batch
146 inference with token truncation enabled.
147
148 ### `format_prediction(text, prediction) -> str`
149 Formats a single result into a human-readable string:
150 `LABEL (score) - original text`.
151
152 ### `load_texts_from_file(path) -> list[str]`
153 Reads a UTF-8 file, strips blank lines, and returns one sentence per
154 non-empty line.
155
156 ### `build_parser() -> ArgumentParser`
157 Constructs the `argparse` parser with `--text` / `-t` and
158 `--input-file` / `-i` flags.
159
160 ### `main() -> None`
161 Entry point: parses args, aggregates texts from both sources,
162 calls inference, and prints results.
163
164 ---
165
166 ## 4. Usage
167
168 ```bash
169 # Inline text
170 python project.py -t "I love this!" "Terrible experience."
171
172 # From file
173 python project.py -i reviews.txt
174
175 # Both sources combined
176 python project.py -t "Great!" -i reviews.txt
177
178 # With HuggingFace token (private models)
179 HF_TOKEN=hf_xxx python project.py -t "Hello world"
180 ```
181
182 **Expected output:**
183 ```
184 POSITIVE (0.9998) - I love this!
185 NEGATIVE (0.9995) - Terrible experience.
186 ```

```